TEST SUMMARY REPORT

Restaurant Table Reservation System

Prepared by

6633014021 Kongpop Boonma

6733065121 Nattarat Samartkit

6733169821 Patcharapon Srisuwan

6733226921 Worachart Poungtabtim

6733288821 Aphiwich Sangpet

6733289421 Apiwit Saengangkhanawin

Course: 2110583 – Practical Software Testing and Quality Assurance

Section: 119

Institution: Chulalongkorn University

Table of Contents

Table of Contents **A**
Project Overview **1**
User Roles and Responsibilities 1
System Requirements and Defined Assumptions 1
Functional Requirements 2
Non-Functional Requirements 4
API Specification 5
Error Responses 10
Test Case Design Specification **11**
API Test Specification Report **105**
Automated Test Specification **175**
Customer Core Flow 175
Successful Registration 175
User Authentication 175
Table Availability Search 175
Create Reservation 175
Critical Business Logic & Constraints 176
Double Booking Prevention 176
Guest Capacity Check 176
Cancellation Lead Time 176
Staff & Admin Operations 177
Manual Walk-in Reservation 177
Live Table Status Update 177
Admin System Configuration 177
Performance Testing Analysis Report **178**
Executive Summary & Objectives 178
Test Methodology & Workload Profile 178
Key Performance Indicators (KPI) Summary 178
Detailed Behavioral Analysis 179
Quantitative Data Summary 179
Behavioral Trends 180
Response Time Graph 180
Conclusion & Recommendations 180
Security Test Specification Report **181**
Security Test Case Matrix 181
Detailed Security Implementations 182
A. Authentication & Password Policy 182
B. Session Management & API Security 182
C. Data Integrity & Ownership (IDOR/XSS/SQLi) 182
Security Conclusion 183
Team Members & Roles **184**

Project Overview

The **Restaurant Table Reservation System (RTS)** is a web-based application designed to allow customers to reserve tables online. It serves three primary user groups: Customers, Staff, and Administrators, ensuring efficient management of reservations and real-time table availability.

Project Repository: https://github.com/HGKXZ/2110583-qa-testing-reservation-system

User Roles and Responsibilities

Role	Description	Key Permissions
Customer	End-user making reservations.	- Register/Login - Create/Modify/Cancel reservations - View reservation history
Staff	Employee managing daily operations.	- View reservations - Create walk-in bookings - Update table status
Admin	System administrator.	- Manage restaurant profile - Manage tables - View reports and statistics

System Requirements and Defined Assumptions

The following functional and non-functional requirements define the scope of the Restaurant Reservation System. To ensure a consistent testing baseline, certain architectural and business logic assumptions have been established. Notably, the system is modeled as a **Buffet-style service**, where table availability is managed through pre-defined time slots rather than a variable A La Carte duration. These assumptions serve as the foundation for the test design and performance benchmarking documented in this report.

Functional Requirements

Account Management

FR-01 Registration: The system shall allow customers to register using
- Name
- Email (unique)
- Password
- Phone number (optional)

FR-02 Login:
- The system shall authenticate users using email and password.
- The system shall display an error message for invalid credentials.

FR-03 Password Management: The system shall allow users to perform a password reset routine to recover account access.

Restaurant Search & Discovery

FR-04 Search Functionality: The system shall allow users to search for restaurants based on restaurant type or name.

Reservation Management

FR-05 View Availability: The system shall display available tables based on
- Date
- Time
- Number of guests

FR-06 Make Reservation: The system shall allow customers to create a reservation including
- Date
- Time
- Number of guests
- Optional special request

Validation rules:
- Past dates shall not be allowed.
- Reservation must be within operating hours.
- Guest count must not exceed table capacity.

FR-07 Reservation Confirmation:
- The system shall display confirmation immediately after successful booking.
- The system shall store reservation data in the Reservation Database.

FR-08 Modify Reservation: Customers shall modify reservations before the reservation time.

FR-09 Cancel Reservation:
- Customers may cancel reservations at least 1 hour before reservation time.
- The system shall automatically update table availability after cancellation.

FR-10 Automated Cancellation: The system shall automatically cancel a booking and release the table if the customer does not check in within 15 minutes of the reserved time.

FR-11 Notification: The system shall notify staff via dashboard notification when:
- a reservation is created
- a reservation is modified
- a reservation is fulfilled
- a reservation is cancelled

Table Management

FR-12 Table Status: Staff shall update table status as:
- Available
- Occupied
- Checked-In
- Out of Service.

FR-13 Double Booking Prevention: The system shall prevent multiple reservations for the same table and time slot.

Walk-in Reservation

FR-14 Manual Reservation: Staff shall create reservations for walk-in customers.

Restaurant Profile Management

FR-15 Manage Restaurant Information: Admin shall update:
- Table configurations
- Price per person
- Operating hours
- Restaurant name
- Contact information

Information shall be stored in the Restaurant Database.

Non-Functional Requirements

Performance Requirements

- Response Time Requirement

API	Max Response Time
POST /auth/login	≤ 1 second
POST /reservations	≤ 2 seconds
GET /reservations/my	≤ 1.5 seconds
DELETE /reservations/{id}	≤ 1 second
GET /tables	≤ 1 second

- Concurrent Users

System must support: 1000 concurrent users

- Error Rate

Error rate must be < 1%

- Duration

System should run 1 hour smoothly

Security Requirements

- Password Policy
- Login Attempt Limitation
- Token Expiration
- Input Validation Security
- API Security
- Sensitive data protection

API Specification

Base Information

- Base URL: https://api.restaurant-reservation.com/v1
- Data Format: Request and Response use JSON
- Authentication: Bearer Token (JWT)

Authentication APIs

Register Customer

Method: POST
Endpoint: /auth/register
Request Example:

```
{
  "name": "John Smith",
  "email": "john@email.com",
  "password": "Password123",
  "phone": "0812345678"
}
```

Response Example:

```
{
  “userId": 101,
  "message": "Registration successful"
}
```

Login Customer

Method: POST
Endpoint: /auth/login
Request Example:

```
{
  "email": "john@email.com",
  "password": "Password123"
}
```

Response Example:

```
{
  "token": "jwt_token_here",
  "role": "Customer"
}
```

Reservation APIs

Create Reservation

Method: POST
Endpoint: /reservations
Request Example:

```
{
  "date": "2026-05-20",
  "time": "18:00",
  "guestCount": 4,
  "specialRequest": "Window seat"
}
```

Response Example:

```
{
  "reservationId": 5001,
  "status": "CONFIRMED"
}
```

Business Rules:

- Guest count must be between 1 and 10
- Reservation must be future date/time
- Table must be available

Cancel Reservation

Method: DELETE
Endpoint: /reservations/{reservationId}
Response Example:

```
{
  "message": "Reservation cancelled"
}
```

Table Management APIs (Staff)

Get Table List

Method: GET
Endpoint: /tables
Response Example:


```
{
  tableId": 1,
  "capacity": 4,
  "status": "AVAILABLE"
}
```

Admin APIs

View All Reservations

Method: GET
Endpoint: /admin/reservations
Response Example:


```
{
  "reservationId": 5001,
  "customerName": "John Smith",
  "tableId": 3,
  "status": "CONFIRMED"
}
```

Error Responses

HTTP Code	Meaning
400	Invalid input data
401	Authentication required
404	Reservation not found
409	Table not available

Test Case Design Specification

TC-001-001: Verify successful registration with valid fields

Functional Requirement (FR): FR-01 Registration
Priority: High
Pre-conditions: User is on Registration page

Input Data

Name: John Doe
Email: tc001001@test.com
Password: Pass1234
Phone: 0812345678

Test Execution Steps

1. Enter valid Name
2. Enter unique Email
3. Enter Phone & Password
4. Click "Register"

Outcome & Validation

- **Expected Result:** System displays "Registration Successful" message
- **Post-conditions:** User record created in DB
- **Pass/Fail Criteria:**
  Pass: Account created
  Fail: Error message shown

TC-001-002: Verify Name field with minimum length

Functional Requirement (FR): FR-01 Registration
Priority: Medium
Pre-conditions: User is on Registration page

Input Data

Name: TC
Email: tc001002@test.com
Pass: Pass1234

Test Execution Steps

1. Enter Name with 2 chars
2. Click "Register"

Outcome & Validation

- **Expected Result:** System accepts name and completes registration
- **Post-conditions:** User record created in DB
- **Pass/Fail Criteria:**
  Pass: Registration successful
  Fail: Input rejected

TC-001-003: Verify Name field with maximum length

Functional Requirement (FR): FR-01 Registration
Priority: Medium
Pre-conditions: User is on Registration page

Input Data

Name: "A" * 60
Email: max@test.com

Test Execution Steps

1. Enter Name with 60 chars
2. Click "Register"

Outcome & Validation

- **Expected Result:** System accepts name and completes registration
- **Post-conditions:** User record created in DB
- **Pass/Fail Criteria:**
  Pass: Registration successful
  Fail: Input rejected

TC-001-004: Verify Name rejects numbers/special characters

Functional Requirement (FR): FR-01 Registration
Priority: Medium
Pre-conditions: User is on Registration page

Input Data

Name: John123
Email: tc001004@test.com
Password: Pass1234

Test Execution Steps

1. Enter Name with numbers/special characters included
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "Letters and spaces only"
- **Post-conditions:** No user created
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-001-005: Verify Email rejects invalid format

Functional Requirement (FR): FR-01 Registration
Priority: High
Pre-conditions: User is on Registration page

Input Data

Email: test.com

Test Execution Steps

1. Enter Email without "@"
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "Must be valid email format"
- **Post-conditions:** No user created
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Form submitted

TC-001-006: Verify Registration fails with duplicate Email

Functional Requirement (FR): FR-01 Registration
Priority: High
Pre-conditions: Account with "dup@test.com" exists

Input Data

Email: dup@test.com

Test Execution Steps

1. Enter Email with duplicated email
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "Email already exists"
- **Post-conditions:** No new record created
- **Pass/Fail Criteria:**
  Pass: Duplicate rejected
  Fail: Created double accounts

TC-001-007: Verify Password with minimum length

Functional Requirement (FR): FR-01 Registration
Priority: High
Pre-conditions: User is on Registration page

Input Data

Pass: Abcdefg1

Test Execution Steps

1. Enter 8-char Password
2. Click "Register"

Outcome & Validation

- **Expected Result:** System accepts and completes registration
- **Post-conditions:** User record created in DB.
- **Pass/Fail Criteria:**
  Pass: Registration successful
  Fail: Rejected 8-char pass

TC-001-008: Verify Password: At least 1 uppercase required

Functional Requirement (FR): FR-01 Registration
Priority: Medium
Pre-conditions: User is on Registration page

Input Data

Pass: password123

Test Execution Steps

1. Enter all lowercase pass
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "At least 1 uppercase required".
- **Post-conditions:** No user created.
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-001-009: Verify Password: At least 1 number required

Functional Requirement (FR): FR-01 Registration
Priority: Medium
Pre-conditions: User is on Registration page

Input Data

Pass: PasswordOnly

Test Execution Steps

1. Enter Password with no digits
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "At least 1 number required"
- **Post-conditions:** No user created.
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-001-010: Verify Phone number with minimum length

Functional Requirement (FR): FR-01 Registration
Priority: Medium
Pre-conditions: User is on Registration page

Input Data

Phone: 0812345678

Test Execution Steps

1. Enter 10-digit Phone No.
2. Click "Register"

Outcome & Validation

- **Expected Result:** System accepts and completes registration
- **Post-conditions:** User record created in DB.
- **Pass/Fail Criteria:**
  Pass: Registration successful
  Fail: Rejected 10-digit phone

TC-001-011: Verify Phone number at maximum length

Functional Requirement (FR): FR-01 Registration
Priority: Medium
Pre-conditions: User is on Registration page

Input Data

Phone: 081234567890123

Test Execution Steps

1. Enter a 15-digit phone number
2. Fill other fields validly
3. Click "Register"

Outcome & Validation

- **Expected Result:** System accepts input and completes registration
- **Post-conditions:** User record created in DB
- **Pass/Fail Criteria:**
  Pass: Registration successful
  Fail: Error or truncation occurs

TC-001-012: Verify Phone number exceeds maximum length

Functional Requirement (FR): FR-01 Registration
Priority: Medium
Pre-conditions: User is on Registration page

Input Data

Phone: 0812345678901234

Test Execution Steps

1. Enter a 16-digit phone number
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "Maximum 15 digits allowed"
- **Post-conditions:** No user created
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-001-013: Verify Phone number below minimum length

Functional Requirement (FR): FR-01 Registration
Priority: Medium
Pre-conditions: User is on Registration page

Input Data

Phone: 081234567

Test Execution Steps

1. Enter a 9-digit phone number
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "Minimum 10 digits required"
- **Post-conditions:** No user created
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-001-014: Verify Phone number rejects non-digit characters

Functional Requirement (FR): FR-01 Registration
Priority: High
Pre-conditions: User is on Registration page

Input Data

Phone: 081-234-5678 or 081234567a

Test Execution Steps

1. Enter phone number containing letters or symbols
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "Digits only"
- **Post-conditions:** No user created
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-002-001: Login with valid credentials

Functional Requirement (FR): FR-02 Login
Priority: High
Pre-conditions: User account is registered and active

Input Data

Email: "test@example.com"
Pass: "Pass1234"

Test Execution Steps

1. Navigate to Login page
2. Enter valid Email
3. Enter valid Password
4. Click "Login"

Outcome & Validation

- **Expected Result:** User is authenticated and redirected to Home/Dashboard
- **Post-conditions:** Active session created
- **Pass/Fail Criteria:**
  Pass: Login successful
  Fail: Access denied or error

TC-002-002: Login denial for Invalid Password

Functional Requirement (FR): FR-02 Login
Priority: High
Pre-conditions: User account is registered

Input Data

Email: "test@example.com"
Pass: "WrongPass99"

Test Execution Steps

1. Enter valid Email
2. Enter incorrect Password
3. Click "Login"

Outcome & Validation

- **Expected Result:** System denies access and shows "Invalid credentials".
- **Post-conditions:** No session created.
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: User gains access

TC-002-003: Login denial for Unregistered Email

Functional Requirement (FR): FR-02 Login
Priority: High
Pre-conditions: Email is not in the database

Input Data

Email: "unknown@mail.com"
Pass: "AnyPass1"

Test Execution Steps

1. Enter unregistered Email
2. Enter any Password
3. Click "Login"

Outcome & Validation

- **Expected Result:** System denies access and shows "Invalid credentials".
- **Post-conditions:** No session created.
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: User gains access

TC-002-004: Verify login attempt with empty fields

Functional Requirement (FR): FR-02 Login
Priority: Medium
Pre-conditions: User is on Login page

Input Data

Email: ""
Pass: ""

Test Execution Steps

1. Leave Email and Password blank
2. Click "Login"

Outcome & Validation

- **Expected Result:** System displays "Please fill in this field" validation.
- **Post-conditions:** No session created.
- **Pass/Fail Criteria:**
  Pass: Warning shown
  Fail: System attempts process

TC-002-005: Verify Case Sensitivity of Password

Functional Requirement (FR): FR-02 Login
Priority: Medium
Pre-conditions: User account exists with "Pass1234"

Input Data

Email: "test@example.com"
Pass: "pass1234"

Test Execution Steps

1. Enter valid Email
2. Enter password in lowercase
3. Click "Login"

Outcome & Validation

- **Expected Result:** System denies access due to case mismatch.
- **Post-conditions:** No session created.
- **Pass/Fail Criteria:**
  Pass: Access denied
  Fail: Login successful

TC-002-006: Verify Logout functionality

Functional Requirement (FR): FR-02 Login
Priority: High
Pre-conditions: User is currently logged in

Input Data

User Session

Test Execution Steps

1. Click on "Logout" button

Outcome & Validation

- **Expected Result:** User is redirected to Login page or Home.
- **Post-conditions:** Session is destroyed.
- **Pass/Fail Criteria:**
  Pass: Session cleared
  Fail: User can still access pages

TC-003-001: Successful password reset

Functional Requirement (FR): FR-03 Password Management
Priority: High
Pre-conditions: User has received a valid reset link/token.

Input Data

New: Pass1234
Confirm: Pass1234

Test Execution Steps

1. Open reset link
2. Enter new valid password
3. Confirm password
4. Click 'Reset'

Outcome & Validation

- **Expected Result:** System displays "Password has been reset successfully."
- **Post-conditions:** User DB updated with new hashed password.
- **Pass/Fail Criteria:**
  Pass: Can login with new PW.Fail: Error shown.

TC-003-002: Attempt to use an Expired Token

Functional Requirement (FR): FR-03 Password Management
Priority: High
Pre-conditions: Token was issued > 24 hours ago.

Input Data

Token: [Expired]

Test Execution Steps

1. Open the expired reset link
2. Enter new password
3. Click 'Reset'

Outcome & Validation

- **Expected Result:** System displays error: "Reset link has expired. Please request a new one."
- **Post-conditions:** Password remains unchanged.
- **Pass/Fail Criteria:**
  Pass: Reset blocked.Fail: Allowed to reset.

TC-003-003: New and Confirm Password Mismatch

Functional Requirement (FR): FR-03 Password Management
Priority: Medium
Pre-conditions: User is on reset page.

Input Data

New: Pass1111
Confirm: Pass2222

Test Execution Steps

1. Enter "Pass1111"
2. Enter "Pass2222" in confirm field
3. Click 'Reset'

Outcome & Validation

- **Expected Result:** System displays error: "Passwords do not match."
- **Post-conditions:** Password remains unchanged.
- **Pass/Fail Criteria:**
  Pass: Error shown.Fail: Form submitted.

TC-003-004: Token Re-use Prevention

Functional Requirement (FR): FR-03 Password Management
Priority: High
Pre-conditions: Token was already used in TC-003-001.

Input Data

Token: [Used]

Test Execution Steps

1. Use the 'Back' button to the reset page
2. Try to reset again with same token

Outcome & Validation

- **Expected Result:** System displays error: "This link has already been used."
- **Post-conditions:** No further change.
- **Pass/Fail Criteria:**
  Pass: Reset blocked.Fail: Multiple resets.

TC-003-005: Verify Password with minimum length

Functional Requirement (FR): FR-03 Password Management
Priority: High
Pre-conditions: User is on reset page.

Input Data

Pass: Abcd123

Test Execution Steps

1. Enter 7-char Password
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "Minimum length: 8 characters"
- **Post-conditions:** Password remains unchanged.
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-003-006: Verify Password with maximum length

Functional Requirement (FR): FR-03 Password Management
Priority: High
Pre-conditions: User is on reset page.

Input Data

Pass: Password["1"*13]

Test Execution Steps

1. Enter 21-char Password
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "Maximum length: 20 characters"
- **Post-conditions:** Password remains unchanged.
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-003-007: Verify Password: At least 1 uppercase required

Functional Requirement (FR): FR-03 Password Management
Priority: Medium
Pre-conditions: User is on reset page.

Input Data

Pass: password123

Test Execution Steps

1. Enter all lowercase pass
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "At least 1 uppercase required".
- **Post-conditions:** Password remains unchanged.
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-003-008: Verify Password: At least 1 number required

Functional Requirement (FR): FR-03 Password Management
Priority: Medium
Pre-conditions: User is on reset page.

Input Data

Pass: PasswordOnly

Test Execution Steps

1. Enter Password with no digits
2. Click "Register"

Outcome & Validation

- **Expected Result:** System displays error: "At least 1 number required"
- **Post-conditions:** Password remains unchanged.
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Registration proceeds

TC-004-S01: Search by Exact Name

Functional Requirement (FR): FR-04 Search Functionality
Priority: High
Pre-conditions: Restaurant "Teenoi" exists in DB.

Input Data

MK Gold

Test Execution Steps

1. Type "Teenoi" in search box
2. Click Search

Outcome & Validation

- **Expected Result:** System displays the card for "Teenoi" only.
- **Post-conditions:** N/A
- **Pass/Fail Criteria:**
  Pass: Correct store shown.
  Fail: No result or wrong store.

TC-004-S02: Search by Category

Functional Requirement (FR): FR-04 Search Functionality
Priority: High
Pre-conditions: "Japanese" category has 3 stores.

Input Data

Type: Japanese

Test Execution Steps

1. Select "Japanese" from Type dropdown
2. Click Search

Outcome & Validation

- **Expected Result:** System displays all 3 Japanese restaurants.
- **Post-conditions:** N/A
- **Pass/Fail Criteria:**
  Pass: List matches DB category.
  Fail: Shows other types.

TC-004-S03: Keyword Search (Partial Match)

Functional Requirement (FR): FR-04 Search Functionality
Priority: Medium
Pre-conditions: Store name "Shabushi" exists.

Input Data

Shabu

Test Execution Steps

1. Type "Shabu" in search box
2. Click Search

Outcome & Validation

- **Expected Result:** System displays "Shabushi" and any other store with "Shabu" in name.
- **Post-conditions:** N/A
- **Pass/Fail Criteria:**
  Pass: Partial match works.
  Fail: Must type full name.

TC-004-S04: Search for non-existent store

Functional Requirement (FR): FR-04 Search Functionality
Priority: Medium
Pre-conditions: No store named "Mars Diner".

Input Data

Mars Diner

Test Execution Steps

1. Type "Mars Diner"
2. Click Search

Outcome & Validation

- **Expected Result:** System displays message: "No restaurants found matching your search."
- **Post-conditions:** N/A
- **Pass/Fail Criteria:**
  Pass: Proper message shown.
  Fail: Error/Empty screen.

TC-004-S05: Reset Search / Clear Filter

Functional Requirement (FR): FR-04 Search Functionality
Priority: Low
Pre-conditions: A search result is currently displayed.

Input Data

N/A

Test Execution Steps

1. Click 'Clear' or 'Show All' button

Outcome & Validation

- **Expected Result:** System clears the search box and displays the full list of restaurants.
- **Post-conditions:** Full list displayed.
- **Pass/Fail Criteria:**
  Pass: UI resets to default.
  Fail: Search term stuck.

TC-005-001: View availability for a valid future date and time

Functional Requirement (FR): FR-05 View Availability
Priority: High
Pre-conditions: Tables are set up in the system

Input Data

Date: Tomorrow
Time: 18:00
Guests: 4

Test Execution Steps

1. Select a future date
2. Select a valid time slot
3. Enter 4 guests
4. Click "Search"

Outcome & Validation

- **Expected Result:** System displays a list of available tables matching the criteria
- **Post-conditions:** Search results are displayed
- **Pass/Fail Criteria:**
  Pass: Tables shown
  Fail: No tables shown or error

TC-005-002: Guest count at minimum boundary

Functional Requirement (FR): FR-05 View Availability
Priority: Medium
Pre-conditions: System has tables with capacity 1 or more

Input Data

Guests: 1

Test Execution Steps

1. Enter 1 in guest count
2. Search for valid date/time

Outcome & Validation

- **Expected Result:** System displays tables that can accommodate 1 person
- **Post-conditions:** Search results are displayed
- **Pass/Fail Criteria:**
  Pass: Valid tables shown
  Fail: Error or no results

TC-005-003: Guest count at maximum boundary

Functional Requirement (FR): FR-05 View Availability
Priority: High
Pre-conditions: System has tables with capacity 10

Input Data

Guests: 10

Test Execution Steps

1. Enter 10 in guest count
2. Search for valid date/time

Outcome & Validation

- **Expected Result:** System displays tables with capacity 10.
- **Post-conditions:** Search results are displayed
- **Pass/Fail Criteria:**
  Pass: Valid tables shown
  Fail: Rejection of 10 guests

TC-005-004: Guest count exceeds maximum

Functional Requirement (FR): FR-05 View Availability
Priority: Medium
Pre-conditions: Max capacity is 10

Input Data

Guests: 11

Test Execution Steps

1. Enter 11 in guest count
2. Search

Outcome & Validation

- **Expected Result:** System displays message: "Maximum 10 guests allowed per table."
- **Post-conditions:** No results shown
- **Pass/Fail Criteria:**
  Pass: Error shown
  Fail: Search proceeds

TC-005-005: Search for a past date

Functional Requirement (FR): FR-05 View Availability
Priority: High
Pre-conditions: Today is April 6, 2026

Input Data

Date: 2026-04-05

Test Execution Steps

1. Select April 5, 2026
2. Click "Search"

Outcome & Validation

- **Expected Result:** System prevents selection or shows: "Must select a future date."
- **Post-conditions:** No results shown
- **Pass/Fail Criteria:**
  Pass: Error/Block shown
  Fail: Allows past search

TC-005-006: Search when all tables are fully booked

Functional Requirement (FR): FR-05 View Availability
Priority: High
Pre-conditions: All tables are already reserved for the chosen slot

Input Data

Date: ValidTime: Busy Slot

Test Execution Steps

1. Enter valid Date, Time, and Guests

Outcome & Validation

- **Expected Result:** System displays: "No tables available for this time."
- **Post-conditions:** No results shown
- **Pass/Fail Criteria:**
  Pass: Correct message shown.Fail: Shows booked tables.

TC-006-001: Create reservation with valid data

Functional Requirement (FR): FR-06 Make Reservation
Priority: High
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Date: 2026-05-20
Time: 18:00
Guests:
4Req: "Window seat"

Test Execution Steps

1. Select a future date
2. Select time
3. Enter 4 guests
4. Enter special request
5. Click "Confirm"

Outcome & Validation

- **Expected Result:** System displays "Reservation Created Successfully" and a Reservation ID
- **Post-conditions:** Reservation status is "PENDING" or "CONFIRMED".
- **Pass/Fail Criteria:**
  Pass: Data saved in DB & ID generated.
  Fail: Error message shown.

TC-006-002: Guest count at Minimum

Functional Requirement (FR): FR-06 Make Reservation
Priority: Medium
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Guests: 1

Test Execution Steps

1. Enter 1 in Guest Count field
2. Fill other valid data
3. Click "Confirm"

Outcome & Validation

- **Expected Result:** System accepts 1 person and proceeds to create reservation
- **Post-conditions:** Reservation record created.
- **Pass/Fail Criteria:**
  Pass: Reservation success.
  Fail: Rejection of 1 guest.

TC-006-003: Guest count at Maximum

Functional Requirement (FR): FR-06 Make Reservation
Priority: High
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Guests: 10

Test Execution Steps

1. Enter 10 in Guest Count field
2. Click "Confirm"

Outcome & Validation

- **Expected Result:** System accepts 10 persons and proceeds
- **Post-conditions:** Reservation record created.
- **Pass/Fail Criteria:**
  Pass: Reservation success.
  Fail: Rejection of 10 guests.

TC-006-004: Guest count below Minimum

Functional Requirement (FR): FR-06 Make Reservation
Priority: Medium
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Guests: 0

Test Execution Steps

1. Enter 0 in Guest Count field
2. Click "Confirm"

Outcome & Validation

- **Expected Result:** System displays error: "Guest count must be at least 1."
- **Post-conditions:** No record created.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: System attempts to process.

TC-006-005: Guest count above Maximum

Functional Requirement (FR): FR-06 Make Reservation
Priority: Medium
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Guests: 11

Test Execution Steps

1. Enter 11 in Guest Count field
2. Click "Confirm"

Outcome & Validation

- **Expected Result:** System displays error: "Max capacity is 10 persons."
- **Post-conditions:** No record created.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: System attempts to process.

TC-006-006: Date is set to Yesterday

Functional Requirement (FR): FR-06 Make Reservation
Priority: High
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Date: [Yesterday]

Test Execution Steps

1. Select yesterday's date from picker
2. Click "Confirm"

Outcome & Validation

- **Expected Result:** System displays error: "Past dates are not allowed."
- **Post-conditions:** No record created.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: Date is accepted.

TC-006-007: Time is before Opening

Functional Requirement (FR): FR-06 Make Reservation
Priority: High
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Time: 09:59

Test Execution Steps

1. Enter 09:59 in Time field
2. Click "Confirm"

Outcome & Validation

- **Expected Result:** System displays error: "Outside operating hours (10:00-21:00)."
- **Post-conditions:** No record created.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: Time is accepted.

TC-006-008: Time is after Closing

Functional Requirement (FR): FR-06 Make Reservation
Priority: High
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Time: 21:01

Test Execution Steps

1. Enter 21:01 in Time field
2. Click "Confirm"

Outcome & Validation

- **Expected Result:** System displays error: "Outside operating hours (10:00-21:00)."
- **Post-conditions:** No record created.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: Time is accepted.

TC-006-009: Special Request exactly 200 chars

Functional Requirement (FR): FR-06 Make Reservation
Priority: Low
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Req: [200 chars]

Test Execution Steps

1. Enter a string of 200 characters in request field
2. Click "Confirm"

Outcome & Validation

- **Expected Result:** System accepts and saves the full request string.
- **Post-conditions:** Record created with full request text.
- **Pass/Fail Criteria:**
  Pass: Reservation success.
  Fail: String truncated or error.

TC-006-010: Special Request exceeds 200 chars

Functional Requirement (FR): FR-06 Make Reservation
Priority: Low
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Req: [201 chars]

Test Execution Steps

1. Enter a string of 201 characters
2. Click "Confirm"

Outcome & Validation

- **Expected Result:** System displays error or prevents typing: "Max 200 characters."
- **Post-conditions:** No record created.
- **Pass/Fail Criteria:**
  Pass: Error shown or input blocked.
  Fail: Saved > 200 chars.

TC-006-011: Valid info but Table is already Booked

Functional Requirement (FR): FR-06 Make Reservation
Priority: High
Pre-conditions: - User is logged in
- User is on Reservation page
- A table is already booked for the selected slot

Input Data

Date: Valid
Time: 12:00 (Busy)

Test Execution Steps

1. Enter valid Date, Time, and Guests for a booked slot

Outcome & Validation

- **Expected Result:** System displays: "No available tables for this time."
- **Post-conditions:** No new record created.
- **Pass/Fail Criteria:**
  Pass: Correct error shown.
  Fail: Overbooking occurs.

TC-006-012: Create reservation without Special Request

Functional Requirement (FR): FR-06 Make Reservation
Priority: Medium
Pre-conditions: - User is logged in
- User is on Reservation page

Input Data

Req: "" (Blank)

Test Execution Steps

1. Fill all required fields
2. Leave Special Request empty
3. Click "Confirm"

Outcome & Validation

- **Expected Result:** System processes reservation successfully.
- **Post-conditions:** Record created with null/empty request.
- **Pass/Fail Criteria:**
  Pass: Success.
  Fail: System requires optional field.

TC-007-001: Verify UI transition after successful booking

Functional Requirement (FR): FR-07 Reservation Confirmation
Priority: High
Pre-conditions: Booking is successful

Input Data

N/A

Test Execution Steps

1. Complete booking process
2. Observe the next screen

Outcome & Validation

- **Expected Result:** System immediately displays "Reservation Successful" with details.
- **Post-conditions:** User is on Success screen.
- **Pass/Fail Criteria:**
  Pass: Screen change detected.
  Fail: UI freezes or stays.

TC-007-002: Verify Reservation ID Generation

Functional Requirement (FR): FR-07 Reservation Confirmation
Priority: High
Pre-conditions: User is on Confirmation page

Input Data

Resv ID from TC-005-001

Test Execution Steps

1. Check the Reservation ID displayed on screen

Outcome & Validation

- **Expected Result:** Database contains a record matching the ID, Date, Time, and Guest count.
- **Post-conditions:** ID is stored in DB.
- **Pass/Fail Criteria:**
  Pass: ID is present and unique.
  Fail: ID is missing.

TC-007-003: Verify Data Persistence (Accuracy)

Functional Requirement (FR): FR-07 Reservation Confirmation
Priority: High
Pre-conditions: Booking is successful

Input Data

Resv ID from TC-005-001

Test Execution Steps

1. Check the 'status' field in the database for the new ID

Outcome & Validation

- **Expected Result:** The status must be "PENDING" or "CONFIRMED" (depending on system config)
- **Post-conditions:** Record is persistent in DB.
- **Pass/Fail Criteria:**
  Pass: 100% data match.
  Fail: Data mismatch.

TC-007-004: Verify Summary Details Accuracy

Functional Requirement (FR): FR-07 Reservation Confirmation
Priority: Medium
Pre-conditions: User is on Confirmation page

Input Data

N/A

Test Execution Steps

1. Review Date, Time, Guests, and Request shown on screen

Outcome & Validation

- **Expected Result:** All displayed summary info must match exactly what the user entered.
- **Post-conditions:** User confirms details.
- **Pass/Fail Criteria:**
  Pass: Summary is correct.
  Fail: Summary is wrong.

TC-008-001: Modify reservation before time starts

Functional Requirement (FR): FR-08 Modify Reservation
Priority: High
Pre-conditions: Reservation exists at 19:00. Current time is 14:00.

Input Data

Guests: 4

Test Execution Steps

1. Access 'Edit Reservation'
2. Change guests from 2 to 4
3. Click 'Update'

Outcome & Validation

- **Expected Result:** System updates details and shows "Modification Successful".
- **Post-conditions:** Status remains "CONFIRMED".
- **Pass/Fail Criteria:**
  Pass: DB reflects new data.
  Fail: Error shown.

TC-008-002: Modify at the exact start time

Functional Requirement (FR): FR-08 Modify Reservation
Priority: High
Pre-conditions: Resv time is 18:00. Current time is 18:00.

Input Data

N/A

Test Execution Steps

1. Try to modify reservation

Outcome & Validation

- **Expected Result:** System shows error: "Cannot modify reservation after it has started."
- **Post-conditions:** No change in DB.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: Allowed to edit.

TC-008-003: Modify 1 minute before start time

Functional Requirement (FR): FR-08 Modify Reservation
Priority: Medium
Pre-conditions: Resv time is 18:00. Current time is 17:59.

Input Data

Guests: 3

Test Execution Steps

1. Quickly modify guests count

Outcome & Validation

- **Expected Result:** System accepts and updates successfully.
- **Post-conditions:** Record updated.
- **Pass/Fail Criteria:**
  Pass: Success.
  Fail: Error shown.

TC-008-004: Modify to an over-capacity count

Functional Requirement (FR): FR-08 Modify Reservation
Priority: High
Pre-conditions: Existing table is T-01 (2 pax).

Input Data

Guests: 4

Test Execution Steps

1. Try to update guests to 4 for T-01

Outcome & Validation

- **Expected Result:** System prompts to "Select a larger table" or shows error.
- **Post-conditions:** No change.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: Allowed 4 in 2-pax.

TC-008-005: Modify a Cancelled reservation

Functional Requirement (FR): FR-08 Modify Reservation
Priority: Medium
Pre-conditions: Reservation status is 'Cancelled'.

Input Data

Resv ID

Test Execution Steps

1. Search for cancelled ID
2. Attempt to edit

Outcome & Validation

- **Expected Result:** System shows error: "Cannot modify a cancelled reservation."
- **Post-conditions:** Status stays Cancelled.
- **Pass/Fail Criteria:**
  Pass: Blocked.
  Fail: Allowed to edit.

TC-009-001: Cancel reservation within allowed time

Functional Requirement (FR): FR-09 Cancel Reservation
Priority: High
Pre-conditions: Resv at 18:00. Current time 16:30.

Input Data

Resv ID

Test Execution Steps

1. Navigate to 'My Booking'
2. Click 'Cancel Reservation'
3. Confirm in popup

Outcome & Validation

- **Expected Result:** System shows "Cancellation Successful".
- **Post-conditions:** Status: "CANCELLED".
- **Pass/Fail Criteria:**
  Pass: Status changed.
  Fail: Error shown.

TC-009-002: Cancel exactly 1 hour before

Functional Requirement (FR): FR-09 Cancel Reservation
Priority: High
Pre-conditions: Resv at 18:00. Current time 17:00.

Input Data

Resv ID

Test Execution Steps

1. Click 'Cancel Reservation'

Outcome & Validation

- **Expected Result:** System processes cancellation successfully.
- **Post-conditions:** Table becomes "Available".
- **Pass/Fail Criteria:**
  Pass: Success.
  Fail: Error shown.

TC-009-003: Cancel 59 mins before

Functional Requirement (FR): FR-09 Cancel Reservation
Priority: High
Pre-conditions: Resv at 18:00. Current time 17:01.

Input Data

Resv ID

Test Execution Steps

1. Click 'Cancel Reservation'

Outcome & Validation

- **Expected Result:** System shows error: "Cannot cancel within 1 hour of the reservation."
- **Post-conditions:** Status remains "CONFIRMED".
- **Pass/Fail Criteria:**
  Pass: Cancellation blocked.
  Fail: Success shown.

TC-009-004: Verify Table Availability after cancellation

Functional Requirement (FR): FR-09 Cancel Reservation
Priority: High
Pre-conditions: Table T-02 was "Reserved".

Input Data

Slot: 18:00

Test Execution Steps

1. Complete TC-007-001
2. Check 'View Availability' for that slot

Outcome & Validation

- **Expected Result:** Table T-02 is now displayed as "Available" for other users.
- **Post-conditions:** Table is free.
- **Pass/Fail Criteria:**
  Pass: Table is bookable.
  Fail: Table still locked.

TC-009-005: Attempt to cancel already cancelled booking

Functional Requirement (FR): FR-09 Cancel Reservation
Priority: Medium
Pre-conditions: Resv ID status is "Cancelled".

Input Data

Resv ID

Test Execution Steps

1. Try to trigger cancel action again via API or Direct URL

Outcome & Validation

- **Expected Result:** System displays: "Reservation is already cancelled."
- **Post-conditions:** No change.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: Success message.

TC-010-001: No cancel at 14 mins 59 secs

Functional Requirement (FR): FR-10 Automated Cancellation
Priority: High
Pre-conditions: Resv at 18:00.
No Check-in.

Input Data

Time: 18:14:59

Test Execution Steps

1. Wait until 18:14:59
2. Check Resv Status

Outcome & Validation

- **Expected Result:** Status remains "Confirmed".
- **Post-conditions:** Resv is still active.
- **Pass/Fail Criteria:**
  Pass: No auto-cancel yet.
  Fail: Cancelled early.

TC-010-002: Auto-cancel exactly at 15 mins

Functional Requirement (FR): FR-10 Automated Cancellation
Priority: High
Pre-conditions: Resv at 18:00.
No Check-in.

Input Data

Time: 18:15:00

Test Execution Steps

1. Wait until 18:15:00
2. Check Resv Status

Outcome & Validation

- **Expected Result:** System updates status to "Cancelled" (Auto).
- **Post-conditions:** Table becomes Available.
- **Pass/Fail Criteria:**
  Pass: Status changed.
  Fail: Still active.

TC-010-003: Verify Table Release after auto-cancel

Functional Requirement (FR): FR-10 Automated Cancellation
Priority: High
Pre-conditions: TC-008-002 is completed.

Input Data

Slot: 18:00+

Test Execution Steps

1. Open 'View Availability' as a new Customer

Outcome & Validation

- **Expected Result:** The table previously reserved is now bookable.
- **Post-conditions:** Table is bookable.
- **Pass/Fail Criteria:**
  Pass: Table is free.
  Fail: Table still locked.

TC-010-004: No auto-cancel if Customer already Checked-in

Functional Requirement (FR): FR-10 Automated Cancellation
Priority: High
Pre-conditions: Customer checked-in at 18:05.

Input Data

Time: 18:06:00

Test Execution Steps

1. Wait until 18:06:00
2. Check Resv Status

Outcome & Validation

- **Expected Result:** Status remains "Occupied" (In-progress).
- **Post-conditions:** Resv is "Occupied".
- **Pass/Fail Criteria:**
  Pass: No auto-cancel.
  Fail: System cancels active table.

TC-011-001: Verify notification for New Reservation

Functional Requirement (FR): FR-11 Notification
Priority: High
Pre-conditions: Staff is on Dashboard.
Customer is on Reservation page.

Input Data

Resv ID

Test Execution Steps

1. Customer completes a valid booking

Outcome & Validation

- **Expected Result:** Staff Dashboard displays: "New Reservation: [ResvID] created."
- **Post-conditions:** Alert is visible in notification bell/list.
- **Pass/Fail Criteria:**
  Pass: Create alert shown.
  Fail: No alert shown.

TC-011-002: Verify notification for Modified Reservation

Functional Requirement (FR): FR-11 Notification
Priority: High
Pre-conditions: Staff is on Dashboard.

Input Data

Resv ID

Test Execution Steps

1. Customer modifies an existing booking

Outcome & Validation

- **Expected Result:** Staff Dashboard displays: "Reservation: [ResvID] has been modified."
- **Post-conditions:** Alert is visible in notification bell/list.
- **Pass/Fail Criteria:**
  Pass: Update alert shown.
  Fail: No alert shown.

TC-011-003: Verify notification for Cancelled Reservation

Functional Requirement (FR): FR-11 Notification
Priority: High
Pre-conditions: Staff is on Dashboard.

Input Data

Resv ID

Test Execution Steps

1. Customer cancels a booking

Outcome & Validation

- **Expected Result:** Staff Dashboard displays: "Reservation: [ResvID] has been cancelled."
- **Post-conditions:** Alert is visible in notification bell/list.
- **Pass/Fail Criteria:**
  Pass: Cancel alert shown.
  Fail: No alert shown.

TC-011-004: Verify notification for Ontime Reservation

Functional Requirement (FR): FR-11 Notification
Priority: High
Pre-conditions: Staff is on Dashboard.

Input Data

Resv ID

Test Execution Steps

1. The time meets reservation time

Outcome & Validation

- **Expected Result:** Staff Dashboard displays: "Reservation: [ResvID] is on time."
- **Post-conditions:** Alert is visible in notification bell/list.
- **Pass/Fail Criteria:**
  Pass: On time alert shown.
  Fail: No alert shown.

TC-011-005: Verify no notification if transaction fails

Functional Requirement (FR): FR-11 Notification
Priority: Medium
Pre-conditions: DB error occurs during customer booking.

Input Data

N/A

Test Execution Steps

1. Customer attempts to book (triggers error)

Outcome & Validation

- **Expected Result:** Staff Dashboard does not show any notification.
- **Post-conditions:** Dashboard remains clean.
- **Pass/Fail Criteria:**
  Pass: No false notification.
  Fail: Ghost notification appears.

TC-011-006: Verify notification details accuracy

Functional Requirement (FR): FR-11 Notification
Priority: Medium
Pre-conditions: Multiple notifications are present.

Input Data

N/A

Test Execution Steps

1. Check the content of each alert

Outcome & Validation

- **Expected Result:** Each alert must show the correct Resv ID and timestamp.
- **Post-conditions:** Staff has correct info.
- **Pass/Fail Criteria:**
  Pass: Content is accurate.
  Fail: Wrong ID or time.

TC-012-001: Change table to Occupied

Functional Requirement (FR): FR-12 Table Status
Priority: High
Pre-conditions: Table status is Available/Out of Service

Input Data

Status: Occupied

Test Execution Steps

1. Select T-02
2. Change status to "Occupied"

Outcome & Validation

- **Expected Result:** Status changes to Occupied.
- **Post-conditions:** Table is Occupied.
- **Pass/Fail Criteria:**
  Pass: Status updated in UI & DB.
  Fail: No change.

TC-012-002: Change table to Available

Functional Requirement (FR): FR-12 Table Status
Priority: High
Pre-conditions: Table status is Occupied/Out of Service

Input Data

Status: Out of Service

Test Execution Steps

1. Select T-02
2. Change status to "Out of Service"

Outcome & Validation

- **Expected Result:** Status changes to Available
  Table T-02 becomes bookable again.
- **Post-conditions:** Table is OOS.
- **Pass/Fail Criteria:**
  Pass: Table is visible in search.
  Fail: No change

TC-012-003: Change table to Out of Service

Functional Requirement (FR): FR-12 Table Status
Priority: Medium
Pre-conditions: Table status is Available/Occupied

Input Data

Status: Available

Test Execution Steps

1. Select T-02
2. Change status back to "Available"

Outcome & Validation

- **Expected Result:** Status changes to Our of Service
  Table disappears from Customer's search.
- **Post-conditions:** Table is Available.
- **Pass/Fail Criteria:**
  Pass: Table is hidden for guests.
  Fail: Still bookable.

TC-013-001: Book the same table in a different time slot

Functional Requirement (FR): FR-13 Double Booking Prevention
Priority: High
Pre-conditions: Table is booked for 18:00-19:00.

Input Data

Table: T-02
Time: 19:00

Test Execution Steps

1. Attempt to book for 19:00-20:00

Outcome & Validation

- **Expected Result:** System allows the reservation.
- **Post-conditions:** Two separate records in DB.
- **Pass/Fail Criteria:**
  Pass: Success message.
  Fail: Blocked.

TC-013-002: Book the same table for the same time slot

Functional Requirement (FR): FR-13 Double Booking Prevention
Priority: High
Pre-conditions: Table is booked for 18:00.

Input Data

Table: T-02
Time: 18:00

Test Execution Steps

1. Another user attempts to book at 18:00

Outcome & Validation

- **Expected Result:** System displays error: "This table is already reserved for the selected time."
- **Post-conditions:** No new record created.
- **Pass/Fail Criteria:**
  Pass: Blocked.
  Fail: Double booking occurs.

TC-013-003: Overlap Test (Partial overlap)

Functional Requirement (FR): FR-13 Double Booking Prevention
Priority: High
Pre-conditions: Table is booked 12:00-14:00.

Input Data

Time: 13:00

Test Execution Steps

1. Attempt to book for 13:00-15:00

Outcome & Validation

- **Expected Result:** System blocks the request due to 1-hour overlap.
- **Post-conditions:** Status remains Reserved.
- **Pass/Fail Criteria:**
  Pass: Blocked.
  Fail: Allowed overlap.

TC-013-004: UI Check: Table status reflected on Grid

Functional Requirement (FR): FR-13 Double Booking Prevention
Priority: Medium
Pre-conditions: Table is Reserved.

Input Data

N/A

Test Execution Steps

1. Open 'View Availability' for that slot

Outcome & Validation

- **Expected Result:** Table T-05 is shown as "Reserved" (Red badge) and is unclickable.
- **Post-conditions:** No state change.
- **Pass/Fail Criteria:**
  Pass: UI prevents selection.
  Fail: Clickable.

TC-013-005: Simultaneous booking attempt

Functional Requirement (FR): FR-13 Double Booking Prevention
Priority: High
Pre-conditions: Two users click 'Confirm' at the exact same time.

Input Data

Table: T-01

Test Execution Steps

1. Run automated script to send
2 requests for T-01

Outcome & Validation

- **Expected Result:** Only 1 user succeeds; the other gets "Already Booked" error.
- **Post-conditions:** 1 record only in DB.
- **Pass/Fail Criteria:**
  Pass: No duplicate IDs for T-01.
  Fail: Both succeed.

TC-014-001: Standard Walk-in Creation

Functional Requirement (FR): FR-14 Manual Reservation
Priority: High
Pre-conditions: Staff logged in
Table T-02 is Available.

Input Data

Table: T-02
Guests: 2

Test Execution Steps

1. Click 'Manual Reservation'
2. Select T-02 & Enter 2 Guests
3. Confirm

Outcome & Validation

- **Expected Result:** System creates record;
  Table status becomes Occupied.
- **Post-conditions:** Table T-02 status is OCCUPIED.
- **Pass/Fail Criteria:**
  Pass: Status in DB = Occupied.
  Fail: Stays Available.

TC-014-002: Manual Booking on "Reserved" Table

Functional Requirement (FR): FR-14 Manual Reservation
Priority: High
Pre-conditions: T-03 is already "Reserved"

Input Data

Table: T-03

Test Execution Steps

1. Staff attempts to select T-03 for Walk-in

Outcome & Validation

- **Expected Result:** System shows warning: "Table is reserved for an online booking."
- **Post-conditions:** Table T-03 status remains RESERVED.
- **Pass/Fail Criteria:**
  Pass: Selection blocked.
  Fail: Overbooking allowed.

TC-014-003: Capacity Check (Max Boundary)

Functional Requirement (FR): FR-14 Manual Reservation
Priority: Medium
Pre-conditions: Staff selects T-05 (8 pax cap)

Input Data

Guests: 9

Test Execution Steps

1. Enter 9 Guests (Max+1)
2. Click Confirm

Outcome & Validation

- **Expected Result:** System displays: "Guest count exceeds table capacity."
- **Post-conditions:** No record created; Table remains AVAILABLE.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: Accepted.

TC-014-004: Transition to "Out of Service" mid-process

Functional Requirement (FR): FR-14 Manual Reservation
Priority: Medium
Pre-conditions: T-01 is marked OOS by Admin

Input Data

Table: T-01

Test Execution Steps

1. Staff starts manual booking for T-01
2. Admin sets T-01 to OOS
3. Staff clicks Confirm

Outcome & Validation

- **Expected Result:** System displays: "Table is no longer available for service."
- **Post-conditions:** Table T-01 status remains OUT OF SERVICE.
- **Pass/Fail Criteria:**
  Pass: Transaction fails.
  Fail: Booked on broken table.

TC-014-005: Cross-Role Visibility

Functional Requirement (FR): FR-14 Manual Reservation
Priority: High
Pre-conditions: Staff completes a Manual Booking for T-02

Input Data

Time: Same Slot

Test Execution Steps

1. Switch to 'Customer' role
2. Search for the same time slot

Outcome & Validation

- **Expected Result:** Table T-02 is now shown as "Occupied" or "Unavailable" to the Customer.
- **Post-conditions:** Table T-02 is UNAVAILABLE for online users.
- **Pass/Fail Criteria:**
  Pass: Real-time update.
  Fail: Customer still sees T-02.

TC-015-001: Update basic restaurant info

Functional Requirement (FR): FR-15 Manage Restaurant Information
Priority: High
Pre-conditions: Admin is logged in and on 'Settings' page.

Input Data

Name: "New RTS"
Tel: 0991234567

Test Execution Steps

1. Change Restaurant Name
2. Update Contact Info
3. Click 'Save'

Outcome & Validation

- **Expected Result:** System displays "Settings Updated Successfully".
- **Post-conditions:** Restaurant DB updated with new name/contact.
- **Pass/Fail Criteria:**
  Pass: UI reflects new name.
  Fail: Data not saved.

TC-015-002: Update Price per Person with decimal

Functional Requirement (FR): FR-15 Manage Restaurant Information
Priority: Medium
Pre-conditions: Admin on Settings page.

Input Data

Price: 499.50

Test Execution Steps

1. Enter price with decimal2. Click 'Save'

Outcome & Validation

- **Expected Result:** System accepts and stores the decimal value.
- **Post-conditions:** Price is updated to 499.50.
- **Pass/Fail Criteria:**
  Pass: DB stores float value.
  Fail: Value rounded or error.

TC-015-003: Update Operating Hours (Cross-midnight)

Functional Requirement (FR): FR-15 Manage Restaurant Information
Priority: High
Pre-conditions: Admin on Settings page.

Input Data

Start: 18:00
End: 02:00

Test Execution Steps

1. Set Start Time: 18:002. Set End Time: 02:00 (Next day)

Outcome & Validation

- **Expected Result:** System accepts valid time range even across midnight.
- **Post-conditions:** Hours updated in DB.
- **Pass/Fail Criteria:**
  Pass: Valid range saved.
  Fail: Error "End must be > Start".

TC-015-004: Update Table Configuration (Add Table)

Functional Requirement (FR): FR-15 Manage Restaurant Information
Priority: High
Pre-conditions: Current tables: T-01 to T-05.

Input Data

Table: T-06
Cap: 4

Test Execution Steps

1. Add new table 'T-06'2. Set Capacity: 43. Click 'Save'

Outcome & Validation

- **Expected Result:** New table appears in 'View Availability' grid.
- **Post-conditions:** Table List in DB increased by 1.
- **Pass/Fail Criteria:**
  Pass: T-06 is bookable.
  Fail: T-06 not found.

TC-015-005: Update Table Count with negative value

Functional Requirement (FR): FR-15 Manage Restaurant Information
Priority: Medium
Pre-conditions: Admin on Settings page.

Input Data

Table Count: -5

Test Execution Steps

1. Enter -5 in Table Count field2. Click 'Save'

Outcome & Validation

- **Expected Result:** System shows error: "Table count must be a positive number."
- **Post-conditions:** No change to DB.
- **Pass/Fail Criteria:**
  Pass: Error shown.
  Fail: DB updated with -5.

TC-015-006: Prevent Staff from accessing Admin Settings

Functional Requirement (FR): FR-15 Manage Restaurant Information
Priority: High
Pre-conditions: User is logged as 'Staff'.

Input Data

URL: /admin/settings

Test Execution Steps

1. Attempt to access /admin/settings URL

Outcome & Validation

- **Expected Result:** System redirects to Staff Dashboard or shows "Access Denied".
- **Post-conditions:** No change to DB.
- **Pass/Fail Criteria:**
  Pass: Forbidden access.
  Fail: Staff can edit info.

API Test Specification Report

[TC-001-001] : Verify successful registration with valid fields

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Name: John Doe
Email: tc001001@test.com
Password: Pass1234
Phone: 0812345678

Request Body

```
{
  "name": "John Doe",
  "email": "tc001001@test.com",
  "password": "Pass1234",
  "phone": "0812345678"
}
```

Response Body

```
{
  "userId": 100,
  "message": "Registration successful"
}
```

Post-conditions: User record created in DB
Expected Result: System displays "Registration Successful" message
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-001-002] : Verify Name field with minimum length

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Name: TC
Email: tc001002@test.com
Pass: Pass1234

Request Body

```
{
  "name": "TC",
  "email": "tc001002@test.com",
  "password": "Pass1234",
  "phone": "0812345678"
}
```

Response Body

```
{
  "userId": 101,
  "message": "Registration successful"
}
```

Post-conditions: User record created in DB
Expected Result: System accepts name and completes registration
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-001-003] : Verify Name field with maximum length

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Name: "A" * 60
Email: max@test.com

Request Body

```
{
  "name":
"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
A",
  "email": "max@test.com",
  "password": "Pass1234",
  "phone": "0812345678"
}
```

Response Body

```
{
  "userId": 102,
  "message": "Registration successful"
}
```

Post-conditions: User record created in DB
Expected Result: System accepts name and completes registration
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-001-004] : Verify Name rejects numbers/special characters

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Name: John123
Email: tc001004@test.com
Password: Pass1234

Request Body

```
{
  "name": "John123",
  "email": "tc001004@test.com",
  "password": "Pass1234",
  "phone": "0812345678"
}
```

Response Body

```
{
  "userId": 103,
  "message": "Registration successful"
}
```

Post-conditions: No user created
Expected Result: System displays error: "Letters and spaces only"
Expected Status Code: 400 Bad Request
Received Status Code: 201 Created

[TC-001-005] : Verify Email rejects invalid format

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Email: test.com

Request Body

```
{
  "name": "John Doe",
  "email": "test.com",
  "password": "Pass1234",
  "phone": "0812345678"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Invalid email format"
}
```

Post-conditions: No user created
Expected Result: System displays error: "Must be valid email format"
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-001-006] : Verify Registration fails with duplicate Email

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: Account with "dup@test.com" exists

Input Data

Email: dup@test.com

Request Body

```
{
  "name": "John Doe",
  "email": "dup@test.com",
  "password": "Pass1234",
  "phone": "0812345678"
}
```

Response Body

```
{
  "error": "Email already exists"
}
```

Post-conditions: No new record created
Expected Result: System displays error: "Email already exists"
Expected Status Code: 409 Conflict
Received Status Code: 409 Conflict

[TC-001-007] : Verify Password with minimum length

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Pass: Abcdefg1

Request Body

```
{
  "name": "John Doe",
  "email": "tc001007@test.com",
  "password": "Abcdefg1",
  "phone": "0812345678"
}
```

Response Body

```
{
  "userId": 104,
  "message": "Registration successful"
}
```

Post-conditions: User record created in DB.
Expected Result: System accepts and completes registration
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-001-008] : Verify Password: At least 1 uppercase required

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Pass: password123

Request Body

```
{
  "name": "John Doe",
  "email": "tc001008@test.com",
  "password": "password123",
  "phone": "0010080000"
}
```

Response Body

```
{
  "userId": 105,
  "message": "Registration successful"
}
```

Post-conditions: No user created.
Expected Result: System displays error: "At least 1 uppercase required".
Expected Status Code: 400 Bad Request
Received Status Code: 201 Created

[TC-001-009] : Verify Password: At least 1 number required

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Pass: PasswordOnly

Request Body

```
{
  "name": "John Doe",
  "email": "tc001009@test.com",
  "password": "PasswordOnly",
  "phone": "0010090000"
}
```

Response Body

```
{
  "userId": 106,
  "message": "Registration successful"
}
```

Post-conditions: No user created.
Expected Result: System displays error: "At least 1 number required"
Expected Status Code: 400 Bad Request
Received Status Code: 201 Created

[TC-001-010] : Verify Phone number with minimum length

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Phone: 0812345678

Request Body

```
{
  "name": "John Doe",
  "email": "tc001010@test.com",
  "password": "Pass1234",
  "phone": "0812345678"
}
```

Response Body

```
{
  "userId": 107,
  "message": "Registration successful"
}
```

Post-conditions: User record created in DB.
Expected Result: System accepts and completes registration
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-001-011] : Verify Phone number at maximum length

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Phone: 081234567890123

Request Body

```
{
  "name": "John Doe",
  "email": "tc001011@test.com",
  "password": "Pass1234",
  "phone": "081234567890123"
}
```

Response Body

```
{
  "userId": 108,
  "message": "Registration successful"
}
```

Post-conditions: User record created in DB
Expected Result: System accepts input and completes registration
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-001-012] : Verify Phone number exceeds maximum length

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Phone: 0812345678901234

Request Body

```
{
  "name": "John Doe",
  "email": "tc001011@test.com",
  "password": "Pass1234",
  "phone": "0812345678901234"
}
```

Response Body

```
{
  "userId": 109,
  "message": "Registration successful"
}
```

Post-conditions: No user created
Expected Result: System displays error: "Maximum 15 digits allowed"
Expected Status Code: 400 Bad Request
Received Status Code: 201 Created

[TC-001-013] : Verify Phone number below minimum length

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Phone: 081234567

Request Body

```
{
  "name": "John Doe",
  "email": "tc001013@test.com",
  "password": "Pass1234",
  "phone": "081234567"
}
```

Response Body

```
{
  "userId": 110,
  "message": "Registration successful"
}
```

Post-conditions: No user created
Expected Result: System displays error: "Minimum 10 digits required"
Expected Status Code: 400 Bad Request
Received Status Code: 201 Created

[TC-001-014] : Verify Phone number rejects non-digit characters

Endpoint URL: `/v1/auth/register`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Registration page

Input Data

Phone: 081-234-5678 or 081234567a

Request Body

```
{
  "name": "John Doe",
  "email": "tc001014@test.com",
  "password": "Pass1234",
  "phone": "081-234-5678"
}
```

Response Body

```
{
  "userId": 111,
  "message": "Registration successful"
}
```

Post-conditions: No user created
Expected Result: System displays error: "Digits only"
Expected Status Code: 400 Bad Request
Received Status Code: 201 Created

[TC-002-001] : Login with valid credentials

Endpoint URL: `/v1/auth/login`
HTTP Method: POST
Auth Required: No
Pre-conditions: User account is registered and active

Input Data

Email: "test@example.com"
Pass: "Pass1234"

Request Body

```
{
  "email": "test@example.com",
  "password": "Pass1234"
}
```

Response Body


```
{
  "token":
"fake.eyJ1c2VySWQiOjQsInJvbGUiOiJDdXN0b21lciIsImV4cCI6MTc3NjQ0NzMwMjk5OX0
=.token",
  "role": "Customer"
}
```


Post-conditions: Active session created
Expected Result: User is authenticated and redirected to Home/Dashboard
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-002-002] : Login denial for Invalid Password

Endpoint URL: `/v1/auth/login`
HTTP Method: POST
Auth Required: No
Pre-conditions: User account is registered

Input Data

Email: "test@example.com"
Pass: "WrongPass99"

Request Body

```
{
  "email": "test@example.com",
  "password": "WrongPass99"
}
```

Response Body

```
{
  "error": "Invalid email or password"
}
```

Post-conditions: No session created.
Expected Result: System denies access and shows "Invalid credentials".
Expected Status Code: 401 Unauthorized
Received Status Code: 401 Unauthorized

[TC-002-003] : Login denial for Unregistered Email

Endpoint URL: `/v1/auth/login`
HTTP Method: POST
Auth Required: No
Pre-conditions: Email is not in the database

Input Data

Email: "unknown@mail.com"
Pass: "AnyPass1"

Request Body

```
{
  "email": "unknown@mail.com",
  "password": "AnyPass1"
}
```

Response Body

```
{
  "error": "Invalid email or password"
}
```

Post-conditions: No session created.
Expected Result: System denies access and shows "Invalid credentials".
Expected Status Code: 401 Unauthorized
Received Status Code: 401 Unauthorized

[TC-002-004] : Verify login attempt with empty fields

Endpoint URL: `/v1/auth/login`
HTTP Method: POST
Auth Required: No
Pre-conditions: User is on Login page

Input Data

Email: "
Pass: "

Request Body

```
{
  "email": ",
  "password": "
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "email and password are required"
}
```

Post-conditions: No session created.
Expected Result: System displays "Please fill in this field" validation.
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-002-005] : Verify Case Sensitivity of Password

Endpoint URL: `/v1/auth/login`
HTTP Method: POST
Auth Required: No
Pre-conditions: User account exists with "Pass1234"

Input Data

Email: "test@example.com"
Pass: "pass1234"

Request Body

```
{
  "email": "test@example.com",
  "password": "pass1234"
}
```

Response Body

```
{
  "error": "Invalid email or password"
}
```

Post-conditions: No session created.
Expected Result: System denies access due to case mismatch.
Expected Status Code: 401 Unauthorized
Received Status Code: 401 Unauthorized

[TC-002-006] : Verify Logout functionality

Endpoint URL: `/v1/auth/login`
HTTP Method: POST
Auth Required: Yes
Pre-conditions: User is currently logged in

Input Data

User Session

Request Body

```
N/A
```

Response Body

```
{
  "message": "Logged out successfully"
}
```

Post-conditions: Session is destroyed.
Expected Result: User is redirected to Login page or Home.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-003-001] : Successful password reset

Endpoint URL: `/v1/auth/login`
HTTP Method: POST
Auth Required: Yes
Pre-conditions: User has received a valid reset link/token.

Input Data

New: Pass1234
Confirm: Pass1234

Request Body

```
{
  "email" : "{{customerEmail}}",
  "newPassword" : "Pass1234",
  "confirmPassword" : "Pass1234"
}
```

Response Body

```
{
  "message": "If this email exists, a reset link has been sent"
}
```

Post-conditions: User DB updated with new hashed password.
Expected Result: System displays "Password has been reset successfully."
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-003-002] : Attempt to use an Expired Token

Endpoint URL: `/v1/tables`
HTTP Method: GET
Auth Required: Yes
Pre-conditions: Token was issued > 24 hours ago.

Input Data

Token: [Expired]

Request Body

```
N/A
```

Response Body

```
{
  "error": "Unauthorized",
  "details": "Token has expired"
}
```

Post-conditions: Password remains unchanged.
Expected Result: System displays error: "Reset link has expired. Please request a new one."
Expected Status Code: 401 Unauthorized
Received Status Code: 401 Unauthorized

[TC-003-003] : New and Confirm Password Mismatch

Endpoint URL: `/v1/auth/reset-password`
HTTP Method: POST
Auth Required: Yes
Pre-conditions: User is on reset page.

Input Data

New: Pass1111
Confirm: Pass2222

Request Body

```
{
  "token" : "{{token}}",
  "newPassword" : "Pass1111",
  "confirmPassword" : "Pass2222"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Passwords do not match"
}
```

Post-conditions: Password remains unchanged.
Expected Result: System displays error: "Passwords do not match."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-003-004] : Token Re-use Prevention

Endpoint URL: `/v1/auth/reset-password`
HTTP Method: POST
Auth Required: Yes
Pre-conditions: Token was already used in TC-003-001.

Input Data

Token: [Used]

Request Body

```
{
  "token" : "{{token}}",
  "newPassword" : "Pass1234",
  "confirmPassword" : "Pass1234"
}
```

Response Body

```
{
  "error": "Token has already been used"
}
```

Post-conditions: No further change.
Expected Result: System displays error: "This link has already been used."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-003-005] : Verify Password with minimum length

Endpoint URL: `/v1/auth/reset-password`
HTTP Method: POST
Auth Required: Yes
Pre-conditions: User is on reset page.

Input Data

Pass: Abcd123

Request Body

```
{
  "token" : "{{token}}",
  "newPassword" : "Abcd123",
  "confirmPassword" : "Abcd123"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Password must be at least 8 characters"
}
```

Post-conditions: Password remains unchanged.
Expected Result: System displays error: "Minimum length: 8 characters"
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-003-006] : Verify Password with maximum length

Endpoint URL: `/v1/auth/reset-password`
HTTP Method: POST
Auth Required: Yes
Pre-conditions: User is on reset page.

Input Data

Pass: Password["1"*13]

Request Body

```
{
  "token" : "{{token}}",
  "newPassword" : "AAAAAAAAAAAA1",
  "confirmPassword" : "AAAAAAAAAAAA1"
}
```

Response Body

```
{
  "message": "Password reset successful"
}
```

Post-conditions: Password remains unchanged.
Expected Result: System displays error: "Maximum length: 20 characters"
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-003-007] : Verify Password: At least 1 uppercase required

Endpoint URL: `/v1/auth/reset-password`
HTTP Method: POST
Auth Required: Yes
Pre-conditions: User is on reset page.

Input Data

Pass: password123

Request Body

```
{
  "token" : "{{token}}",
  "newPassword" : "password123",
  "confirmPassword" : "password123"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Password must contain at least 1 uppercase letter"
}
```

Post-conditions: Password remains unchanged.
Expected Result: System displays error: "At least 1 uppercase required".
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-003-008] : Verify Password: At least 1 number required

Endpoint URL: `/v1/auth/reset-password`
HTTP Method: POST
Auth Required: Yes
Pre-conditions: User is on reset page.

Input Data

Pass: PasswordOnly

Request Body

```
{
  "token" : "{{token}}",
  "newPassword" : "PasswordOnly",
  "confirmPassword" : "PasswordOnly"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Password must contain at least 1 number"
}
```

Post-conditions: Password remains unchanged.
Expected Result: System displays error: "At least 1 number required"
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-004-S01] : Search by Exact Name

Endpoint URL: `/v1/restaurants?keyword=MK Gold`
HTTP Method: GET
Auth Required: No
Pre-conditions: Restaurant "Teenoi" exists in DB.

Input Data

MK Gold

Request Body

```
N/A
```

Response Body


```
[
  {
    "restaurantId": 1,
    "name": "MK Gold",
    "type": "Buffet",
    "cuisine": "Thai Sukiyaki / Shabu",
    "pricePerPerson": 369,
    "currency": "THB",
    "rating": 4.2,
    "reviewCount": 18420,
    "tags": [
      "Family-friendly",
      "Halal-option",
      "Group dining",
      "Sukiyaki",
      "Shabu"
    ]
  }
]
```


Post-conditions: N/A
Expected Result: System displays the card for "Teenoi" only.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-004-S02] : Search by Category

Endpoint URL: `/v1/restaurants?type=Japanese`
HTTP Method: GET
Auth Required: No
Pre-conditions: "Japanese" category has 3 stores.

Input Data

Type: Japanese

Request Body

```
N/A
```

Response Body


```
[
  {
    "restaurantId": 1,
    "name": "MK Gold",
    "type": "Japanese",
    "cuisine": "Thai Sukiyaki / Shabu",
    "pricePerPerson": 369,
    "currency": "THB",
    "rating": 4.2,
    "reviewCount": 18420,
    "tags": [
      "Family-friendly",
      "Halal-option",
      "Group dining",
      "Sukiyaki",
      "Shabu"
    ]
  }
]
```


Post-conditions: N/A
Expected Result: System displays all 3 Japanese restaurants.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-004-S03] : Keyword Search (Partial Match)

Endpoint URL: `/v1/restaurants?tag=Shabu`
HTTP Method: GET
Auth Required: No
Pre-conditions: Store name "Shabushi" exists.

Input Data

Shabu

Request Body

N/A

Response Body


```
[
  {
    "restaurantId": 1,
    "name": "MK Gold",
    "type": "Japanese",
    "cuisine": "Thai Sukiyaki / Shabu",
    "pricePerPerson": 369,
    "currency": "THB",
    "rating": 4.2,
    "reviewCount": 18420,
    "tags": [
      "Family-friendly",
      "Halal-option",
      "Group dining",
      "Sukiyaki",
      "Shabu"
    ]
  },
  {
    "restaurantId": 2,
    "name": "Shabu-X",
    "type": "Buffet",
    "cuisine": "Japanese Shabu Shabu",
    "pricePerPerson": 459,
    "currency": "THB",
    "rating": 4.4,
    "reviewCount": 9870,
    "tags": [
      "Japanese",
      "Group dining",
      "Premium meat",
      "Shabu Shabu",
      "All-you-can-eat"
    ]
```

```
  }
]
```

Post-conditions: N/A
Expected Result: System displays "Shabushi" and any other store with "Shabu" in name.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-004-S04] : Search for non-existent store

Endpoint URL: `/v1/restaurants?name=Mars Diner`
HTTP Method: GET
Auth Required: No
Pre-conditions: No store named "Mars Diner".

Input Data

Mars Diner

Request Body

```
N/A
```

Response Body

```
{
  "message": "No restaurants found",
  "data": []
}
```

Post-conditions: N/A
Expected Result: System displays message: "No restaurants found matching your search."
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-004-S05] : Reset Search / Clear Filter

Endpoint URL: `/v1/restaurants`
HTTP Method: GET
Auth Required: No
Pre-conditions: A search result is currently displayed.

Input Data

N/A

Request Body

```
N/A
```

Response Body


```
[
  {
    "restaurantId": 1,
    "name": "MK Gold",
    "type": "Japanese",
    "cuisine": "Thai Sukiyaki / Shabu",
    "pricePerPerson": 369,
    "currency": "THB",
    "rating": 4.2,
    "reviewCount": 18420,
    "tags": [
      "Family-friendly",
      "Halal-option",
      "Group dining",
      "Sukiyaki",
      "Shabu"
    ]
  },
  {
    "restaurantId": 2,
    "name": "Shabu-X",
    "type": "Buffet",
    "cuisine": "Japanese Shabu Shabu",
    "pricePerPerson": 459,
    "currency": "THB",
    "rating": 4.4,
    "reviewCount": 9870,
    "tags": [
      "Japanese",
      "Group dining",
      "Premium meat",
      "Shabu Shabu",
      "All-you-can-eat"
    ]
```

```
  }
]
```

Post-conditions: Full list displayed.
Expected Result: System clears the search box and displays the full list of restaurants.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-005-001] : View availability for a valid future date and time

Endpoint URL:
`/v1/tables/available?date=2026-04-15&time=18:00&guestCount=2`
HTTP Method: GET
Auth Required: Yes
Pre-conditions: Tables are set up in the system

Input Data

Date: Tomorrow
Time: 18:00
Guests: 4

Request Body

```
N/A
```

Response Body

```
{
  "date": "2026-04-15",
  "time": "18:00",
  "guestCount": 2,
  "availableCount": 4,
  "tables": [
    {
      "tableId": 1,
      "capacity": 2
    },
    {
      "tableId": 3,
      "capacity": 4
    },
    {
      "tableId": 4,
      "capacity": 6
    },
    {
      "tableId": 6,
      "capacity": 10
    }
  ]
}
```

Post-conditions: Search results are displayed
Expected Result: System displays a list of available tables matching the criteria
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-005-002] : Guest count at minimum boundary

Endpoint URL:
`/v1/tables/available?date=2026-04-15&time=18:00&guestCount=1`
HTTP Method: GET
Auth Required: Yes
Pre-conditions: System has tables with capacity 1 or more

Input Data

Guests: 1

Request Body

```
N/A
```

Response Body

```
{
  "date": "2026-04-15",
  "time": "18:00",
  "guestCount": 1,
  "availableCount": 4,
  "tables": [
    {
      "tableId": 1,
      "capacity": 2
    },
    {
      "tableId": 3,
      "capacity": 4
    },
    {
      "tableId": 4,
      "capacity": 6
    },
    {
      "tableId": 6,
      "capacity": 10
    }
  ]
}
```

Post-conditions: Search results are displayed
Expected Result: System displays tables that can accommodate 1 person
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-005-003] : Guest count at maximum boundary

Endpoint URL:
`/v1/tables/available?date=2026-04-15&time=18:00&guestCount=10`
HTTP Method: GET
Auth Required: Yes
Pre-conditions: System has tables with capacity 10

Input Data

Guests: 10

Request Body

```
N/A
```

Response Body

```
{
  "date": "2026-04-15",
  "time": "18:00",
  "guestCount": 10,
  "availableCount": 1,
  "tables": [
    {
      "tableId": 6,
      "capacity": 10
    }
  ]
}
```

Post-conditions: Search results are displayed
Expected Result: System displays tables with capacity 10.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-005-004] : Guest count exceeds maximum

Endpoint URL:
`/v1/tables/available?date=2026-04-15&time=18:00&guestCount=11`
HTTP Method: GET
Auth Required: Yes
Pre-conditions: Max capacity is 10

Input Data

Guests: 11

Request Body

```
N/A
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "guestCount must be between 1 and 10"
}
```

Post-conditions: No results shown
Expected Result: System displays message: "Maximum 10 guests allowed per table."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-005-005] : Search for a past date

Endpoint URL:
`/v1/tables/available?date=2026-04-05&time=18:00&guestCount=2`
HTTP Method: GET
Auth Required: Yes
Pre-conditions: Today is April 6, 2026

Input Data

Date: 2026-04-05

Request Body

```
N/A
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "date and time must be in the future"
}
```

Post-conditions: No results shown
Expected Result: System prevents selection or shows: "Must select a future date."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-005-006] : Search when all tables are fully booked

Endpoint URL:
`/v1/tables/available?date=2026-04-15&time=18:00&guestCount=2`
HTTP Method: GET
Auth Required: Yes
Pre-conditions: All tables are already reserved for the chosen slot

Input Data

Date: ValidTime: Busy Slot

Request Body

```
N/A
```

Response Body

```
{
  "message": "No tables available for the requested time and guest count",
  "data": []
}
```

Post-conditions: No results shown
Expected Result: System displays: "No tables available for this time."
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-006-001] : Create reservation with valid data

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Date: 2026-05-20
Time: 18:00
Guests: 4
Req: "Window seat"

Request Body

```
{
  "date":" 2026-05-20",
  "time": "18:00",
  "guestCount": 4,
  "specialRequest": "Window seat"
}
```

Response Body

```
{
  "reservationId": 5002,
  "tableId": 4,
  "status": "CONFIRMED"
}
```

Post-conditions: Reservation status is "PENDING" or "CONFIRMED".
Expected Result: System displays "Reservation Created Successfully" and a Reservation ID
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-006-002] : Guest count at Minimum

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Guests: 1

Request Body

```
{
  "date": "2026-05-20",
  "time": "18:00",
  "guestCount": 1,
  "specialRequest": "Window seat"
}
```

Response Body

```
{
  "reservationId": 5003,
  "tableId": 1,
  "status": "CONFIRMED"
}
```

Post-conditions: Reservation record created.
Expected Result: System accepts 1 person and proceeds to create reservation
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-006-003] : Guest count at Maximum

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Guests: 10

Request Body

```
{
  "date": "2026-05-20",
  "time": "18:00",
  "guestCount": 10,
  "specialRequest": "Window seat"
}
```

Response Body

```
{
  "reservationId": 5004,
  "tableId": 6,
  "status": "CONFIRMED"
}
```

Post-conditions: Reservation record created.
Expected Result: System accepts 10 persons and proceeds
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-006-004] : Guest count below Minimum

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Guests: 0

Request Body

```
{
  "date": "2026-05-20",
  "time": "18:00",
  "guestCount": 0,
  "specialRequest": "Window seat"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Guest count must be between 1 and 10"
}
```

Post-conditions: No record created.
Expected Result: System displays error: "Guest count must be at least 1."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-006-005] : Guest count above Maximum

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Guests: 11

Request Body

```
{
  "date":" 2026-05-20",
  "time": "18:00",
  "guestCount": 11,
  "specialRequest": "Window seat"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Guest count must be between 1 and 10"
}
```

Post-conditions: No record created.
Expected Result: System displays error: "Max capacity is 10 persons."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-006-006] : Date is set to Yesterday

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Date: [Yesterday]

Request Body

```
{
  "date":" 2026-04-13",
  "time": "18:00",
  "guestCount": 2,
  "specialRequest": "Window seat"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Reservation must be a future date/time"
}
```

Post-conditions: No record created.
Expected Result: System displays error: "Past dates are not allowed."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-006-007] : Time is before Opening

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Time: 09:59

Request Body

```
{
  "date":" 2026-05-16",
  "time": "09:59",
  "guestCount": 2,
  "specialRequest": "Window seat"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Reservation must be within operating hours (10:00 - 21:00)"
}
```

Post-conditions: No record created.
Expected Result: System displays error: "Outside operating hours (10:00-21:00)."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-006-008] : Time is after Closing

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Time: 21:01

Request Body

```
{
  "date": "2026-05-16",
  "time": "21:01",
  "guestCount": 2,
  "specialRequest": "Window seat"
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Reservation must be within operating hours (10:00 - 21:00)"
}
```

Post-conditions: No record created.
Expected Result: System displays error: "Outside operating hours (10:00-21:00)."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-006-009] : Special Request exactly 200 chars

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Req: [200 chars]

Request Body

```
{
  "date":" 2026-05-20",
  "time": "18:00",
  "guestCount": 4,
  "specialRequest":
"AAAA
```

[TC-006-010] : Special Request exceeds 200 chars

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Req: [201 chars]

Request Body

```
{
  "date":" 2026-05-20",
  "time": "18:00",
  "guestCount": 4,
  "specialRequest":
"AAAA
```

[TC-006-012] : Create reservation without Special Request

Endpoint URL: `/v1/reservations`
HTTP Method: POST
Auth Required: Yes
Pre-conditions:
- User is logged in
- User is on Reservation page

Input Data

Req: " (Blank)

Request Body

```
{
  "date":" 2026-05-20",
  "time": "18:00",
  "guestCount": 4,
  "specialRequest": "
}
```

Response Body

```
{
  "reservationId": 5003,
  "tableId": 6,
  "status": "CONFIRMED"
}
```

Post-conditions: Record created with null/empty request.
Expected Result: System processes reservation successfully.
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-008-001] : Modify reservation before time starts

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: PATCH
Auth Required: Yes
Pre-conditions: Reservation exists at 19:00. Current time is 14:00.

Input Data

Guests: 4

Request Body

```
{
  "guestCount": 4
}
```

Response Body

```
{
  "reservationId": 5002,
  "status": "CONFIRMED",
  "tableId": 3
}
```

Post-conditions: Status remains "CONFIRMED".
Expected Result: System updates details and shows "Modification Successful".
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-008-002] : Modify at the exact start time

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: PATCH
Auth Required: Yes
Pre-conditions: Resv time is 18:00. Current time is 18:00.

Input Data

N/A

Request Body

```
{
  "guestCount": 5
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Cannot modify reservation at or after start time"
}
```

Post-conditions: No change in DB.
Expected Result: System shows error: "Cannot modify reservation after it has started."
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-008-003] : Modify 1 minute before start time

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: PATCH
Auth Required: Yes
Pre-conditions: Resv time is 18:00. Current time is 17:59.

Input Data

Guests: 3

Request Body

```
{
  "guestCount": 3
}
```

Response Body

```
{
  "reservationId": 5003,
  "status": "CONFIRMED",
  "tableId": 3
}
```

Post-conditions: Record updated.
Expected Result: System accepts and updates successfully.
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-008-004] : Modify to an over-capacity count

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: PATCH
Auth Required: Yes
Pre-conditions: Existing table is T-01 (2 pax).

Input Data

Guests: 4

Request Body

```
{
  "guestCount": 11
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Guest count must be between 1 and 10"
}
```

Post-conditions: No change.
Expected Result: System prompts to "Select a larger table" or shows error.
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-008-005] : Modify a Cancelled reservation

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: PATCH
Auth Required: Yes
Pre-conditions: Reservation status is 'Cancelled'.

Input Data

Resv ID

Request Body

```
{
  "guestCount": 4
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Cannot modify a cancelled reservation"
}
```

Post-conditions: Status stays Cancelled.
Expected Result: System shows error: "Cannot modify a cancelled reservation."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-009-001] : Cancel reservation within allowed time

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: DELETE
Auth Required: Yes
Pre-conditions: Resv at 18:00. Current time 16:30.

Input Data

Resv ID

Request Body

```
N/A
```

Response Body

```
{
  "message": "Reservation cancelled",
  "reservationId": 5007
}
```

Post-conditions: Status: "CANCELLED".
Expected Result: System shows "Cancellation Successful".
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-009-002] : Cancel exactly 1 hour before

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: DELETE
Auth Required: Yes
Pre-conditions: Resv at 18:00. Current time 17:00.

Input Data

Resv ID

Request Body

```
N/A
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Cancellation must be at least 1 hour before reservation time"
}
```

Post-conditions: Table becomes "Available".
Expected Result: System processes cancellation successfully.
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-009-003] : Cancel 59 mins before

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: DELETE
Auth Required: Yes
Pre-conditions: Resv at 18:00. Current time 17:01.

Input Data

Resv ID

Request Body

```
N/A
```

Response Body

```
{
  "message": "Reservation cancelled",
  "reservationId": 5009
}
```

Post-conditions: Status remains "CONFIRMED".
Expected Result: System shows error: "Cannot cancel within 1 hour of the reservation."
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-009-004] : Verify Table Availability after cancellation

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: DELETE
Auth Required: Yes
Pre-conditions: Table T-02 was "Reserved".

Input Data

Slot: 18:00

Request Body

```
N/A
```

Response Body

```
{
  "availableCount": 4,
  "tables": [
    {
      "tableId": 1,
      "capacity": 2
    },
    {
      "tableId": 3,
      "capacity": 4
    },
    {
      "tableId": 4,
      "capacity": 6
    },
    {
      "tableId": 6,
      "capacity": 10
    }
  ]
}
```

Post-conditions: Table is free.
Expected Result: Table T-02 is now displayed as "Available" for other users.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-009-005] : Attempt to cancel already cancelled booking

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: DELETE
Auth Required: Yes
Pre-conditions: Resv ID status is "Cancelled".

Input Data

Resv ID

Request Body

```
N/A
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "Reservation is already cancelled"
}
```

Post-conditions: No change.
Expected Result: System displays: "Reservation is already cancelled."
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-012-001] : Change table to Occupied

Endpoint URL: `/reservations/{{reservationId}}`
HTTP Method: GET
Auth Required: Yes
Pre-conditions: Table status is Available/Out of Service

Input Data

Status: Occupied

Request Body

```
N/A
```

Response Body

```
{
  "availableCount": 2,
  "tables": [
    {
      "tableId": 4,
      "capacity": 6
    },
    {
      "tableId": 6,
      "capacity": 10
    }
  ]
}
```

Post-conditions: Table is Occupied.
Expected Result: Status changes to Occupied.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-013-001] : Book the same table in a different time slot

Endpoint URL: `/reservations`
HTTP Method: PUT
Auth Required: Yes
Pre-conditions: Table is booked for 18:00-19:00.

Input Data

Table: T-02
Time: 19:00

Request Body

```
{
  "date":" 2026-05-30",
  "time": "19:00",
  "guestCount": 2,
  "specialRequest": "Window seat"
}
```

Response Body

```
{
  "reservationId": 5004,
  "tableId": 1,
  "status": "CONFIRMED"
}
```

Post-conditions: Two separate records in DB.
Expected Result: System allows the reservation.
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-015-001] : Update basic restaurant info

Endpoint URL: `/admin/settings`
HTTP Method: PATCH
Auth Required: Yes
Pre-conditions: Admin is logged in and on 'Settings' page.

Input Data

Name: "New RTS"
Tel: 0991234567

Request Body

```
{
  "name": "New RTS",
  "tel": "0991234567"
}
```

Response Body

```
{
  "message": "Settings updated",
  "settings": {
    "name": "New RTS",
    "address": "123 Main Street, Bangkok",
    "pricePerPerson": 299,
    "operatingHours": {
      "open": 10,
      "close": 21
    },
    "tableCount": 6
  }
}
```

Post-conditions: Restaurant DB updated with new name/contact.
Expected Result: System displays "Settings Updated Successfully".
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-015-002] : Update Price per Person with decimal

Endpoint URL: `/admin/settings`
HTTP Method: PATCH
Auth Required: Yes
Pre-conditions: Admin on Settings page.

Input Data

Price: 499.50

Request Body

```
{
  "pricePerPerson": 499.50
}
```

Response Body

```
{
  "message": "Settings updated",
  "settings": {
    "name": "New RTS",
    "address": "123 Main Street, Bangkok",
    "pricePerPerson": 499.5,
    "operatingHours": {
      "open": 10,
      "close": 21
    },
    "tableCount": 6
  }
}
```

Post-conditions: Price is updated to 499.50.
Expected Result: System accepts and stores the decimal value.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-015-003] : Update Operating Hours (Cross-midnight)

Endpoint URL: `/admin/settings`
HTTP Method: PATCH
Auth Required: Yes
Pre-conditions: Admin on Settings page.

Input Data

Start: 18:00
End: 02:00

Request Body

```
{
  "operatingHours": {
      "open": 18,
      "close": 2
    }
}
```

Response Body

```
{
  "message": "Settings updated",
  "settings": {
    "name": "New RTS",
    "address": "123 Main Street, Bangkok",
    "pricePerPerson": 499.5,
    "operatingHours": {
      "open": 18,
      "close": 2
    },
    "tableCount": 6
  }
}
```

Post-conditions: Hours updated in DB.
Expected Result: System accepts valid time range even across midnight.
Expected Status Code: 200 OK
Received Status Code: 200 OK

[TC-015-004] : Update Table Configuration (Add Table)

Endpoint URL: `/tables`
HTTP Method: POST
Auth Required: Yes
Pre-conditions: Current tables: T-01 to T-05.

Input Data

Table: T-06
Cap: 4

Request Body

```
{
  "capacity": 4
}
```

Response Body

```
{
  "tableId": 7,
  "capacity": 4,
  "status": "AVAILABLE"
}
```

Post-conditions: Table List in DB increased by 1.
Expected Result: New table appears in 'View Availability' grid.
Expected Status Code: 201 Created
Received Status Code: 201 Created

[TC-015-005] : Update Table Count with negative value

Endpoint URL: `/admin/settings`
HTTP Method: PATCH
Auth Required: Yes
Pre-conditions: Admin on Settings page.

Input Data

Table Count: -5

Request Body

```
{
  "tableCount": -5
}
```

Response Body

```
{
  "error": "Invalid input data",
  "details": "tableCount must be a non-negative integer"
}
```

Post-conditions: No change to DB.
Expected Result: System shows error: "Table count must be a positive number."
Expected Status Code: 400 Bad Request
Received Status Code: 400 Bad Request

[TC-015-006] : Prevent Staff from accessing Admin Settings

Endpoint URL: `/admin/settings`
HTTP Method: GET
Auth Required: Yes
Pre-conditions: User is logged as 'Staff'.

Input Data

URL: /admin/settings

Request Body

```
N/A
```

Response Body

```
{
  "error": "Forbidden: insufficient permissions"
}
```

Post-conditions: No change to DB.
Expected Result: System redirects to Staff Dashboard or shows "Access Denied".
Expected Status Code: 401 Unauthorized
Received Status Code: 401 Unauthorized

Automated Test Specification

Customer Core Flow

Successful Registration

Verify a customer can register with valid name, unique email, phone, and a strong password.

Test Case ID
TC-001-001 - Verify successful registration with valid fields

User Authentication

Verify successful login with correct credentials and error handling for invalid emails/passwords.

Test Case ID
TC-002-001 - Login with valid credentials
TC-002-002 - Login denial for Invalid Password
TC-002-003 - Login denial for Unregistered Email

Table Availability Search

Ensure the system filters tables correctly based on date, time, and the number of guests.

Test Case ID
TC-005-001 - View availability for a valid future date and time
TC-005-002 - Guest count at minimum boundary
TC-005-003 - Guest count at maximum boundary
TC-005-004 - Guest count exceeds maximum
TC-005-005 - Search for a past date
TC-005-006 - Search when all tables are fully booked

Create Reservation

Complete a full booking for a future date and verify immediate confirmation display.

Test Case ID
TC-006-001 - Create reservation with valid data
TC-007-001 - Verify UI transition after successful booking

Critical Business Logic & Constraints

Double Booking Prevention

Verify the system blocks a second reservation for the same table and time slot.

Test Case ID
TC-013-001 - Book the same table in a different time slot
TC-013-002 - Book the same table for the same time slot

Guest Capacity Check

Ensure a reservation fails if the guest count exceeds the selected table's maximum capacity.

Test Case ID
TC-005-004 - Guest count exceeds maximum

Cancellation Lead Time

Verify system automatically cancels if the user is late ≥ 15 mins after the booking time, and not cancel if the user has already checked-in.

Test Case ID
TC-010-001 - No cancel at 14 mins 59 secs
TC-010-002 - Auto-cancel exactly at 15 mins
TC-010-004 - No auto-cancel if Customer already Checked-in

Staff & Admin Operations

Manual Walk-in Reservation

Verify staff can create reservations for walk-in customers without prior account registration.

Test Case ID
TC-014-001 - Standard Walk-in Creation

Live Table Status Update

Verify staff can change a table's status (e.g., from Available to Occupied) and see it reflected in the system.

Test Case ID
TC-014-001 - Standard Walk-in Creation

Admin System Configuration

Ensure the Admin can update operating hours and restaurant information successfully.

Test Case ID
TC-015-001 - Update basic restaurant info
TC-015-003 - Update Operating Hours (Cross-midnight)

Performance Testing Analysis Report

Executive Summary & Objectives

The primary objective of this assessment was to evaluate the system's stability and scalability under peak load conditions, as specified in the System Requirements Specification (SRS). The testing focused on determining whether the API infrastructure could sustain a high volume of concurrent traffic without performance degradation or service interruption.

- **Testing Tool:** Apache JMeter (v5.6.3)
- **Environment:** Mockoon API Simulation (Localhost)
- **Latency Configuration:** Variable latency was applied per endpoint to simulate real-world network conditions and processing overhead.

Test Methodology & Workload Profile

The test was executed based on a "Stress-to-Steady" profile to simulate peak operational hours.

- **Concurrent Users:** 1,000 Threads (Virtual Users)
- **Ramp-up Period:** 300 Seconds (5 minutes) – designed for gradual load increment.
- **Test Duration:** 1 Hour (3,600 seconds) of sustained peak load.
- **Pacing & Think Time:** A **Constant Timer of 2,000ms** was implemented between requests to simulate authentic human interaction patterns and prevent artificial request flooding.

Key Performance Indicators (KPI) Summary

The execution resulted in a total of **1.7 million samples** processed. The data below summarizes the aggregate performance metrics:

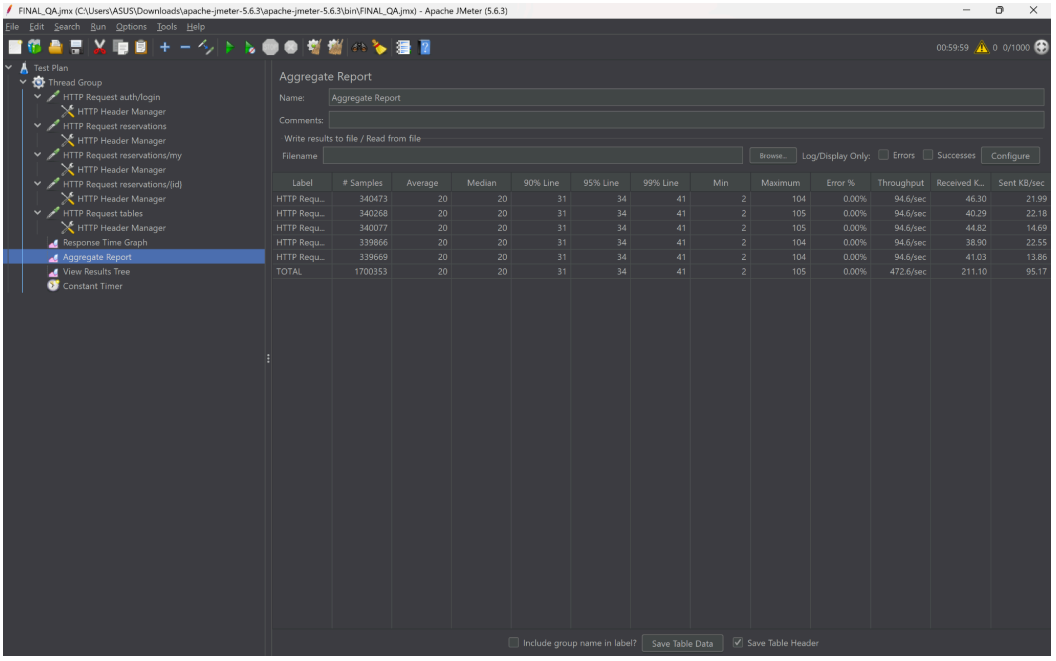
Metric	Results	Target / Threshold	Status
Avg. Response Time	**~20 ms**	< 1,000 - 2,000 ms	**PASS**
Error Rate	**0.00%**	< 1.00%	**PASS**
Throughput	**472.6 req/sec**	N/A	**Optimal**

Detailed Behavioral Analysis

The performance behavior was monitored across three distinct phases:

- **Initial Initialization:** High volatility was observed during the first few minutes as the system allocated resources to handle the initial wave of connections.
- **Ramp-up Progression:** As the thread count scaled toward 1,000, the Response Time showed a marginal and controlled increase from **10–14ms** to approximately **18–20ms**. This indicates a healthy scaling factor.
- **Steady State:** After the first 20 minutes, the system achieved "Equilibrium." Response times remained consistent between **20–22ms** for the remainder of the hour, demonstrating high architectural stability under sustained pressure.

Quantitative Data Summary



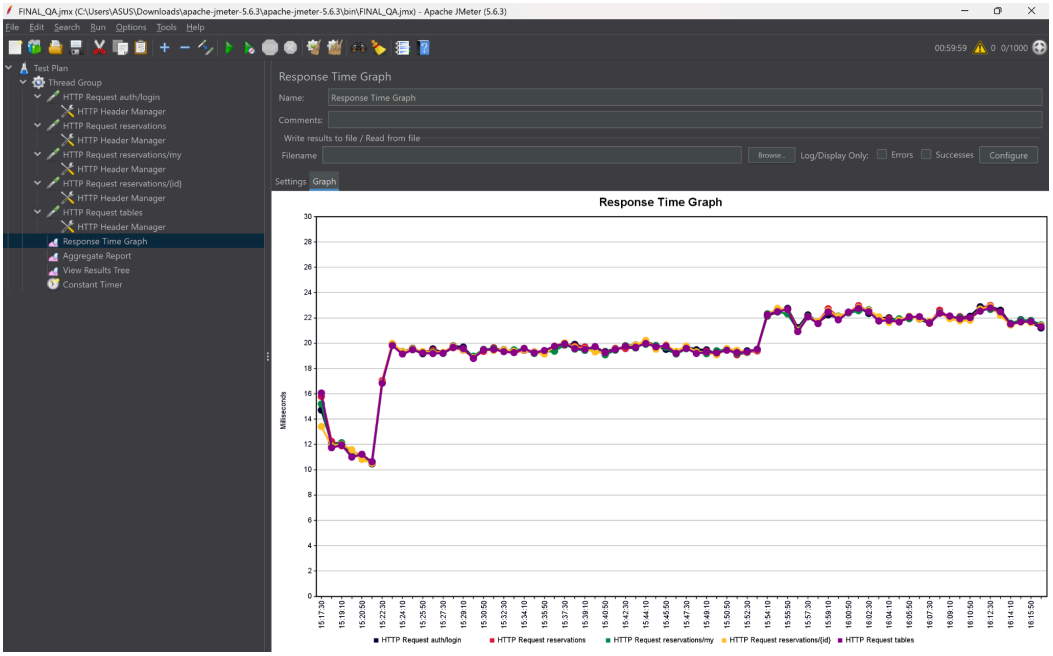
Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received K...	Sent KB/sec
HTTP Requ...	340473	20	20	31	34	41	2	104	0.00%	94.6/sec	46.30	21.99
HTTP Requ...	340268	20	20	31	34	41	2	105	0.00%	94.6/sec	40.29	22.18
HTTP Requ...	340077	20	20	31	34	41	2	105	0.00%	94.6/sec	44.82	14.69
HTTP Requ...	339866	20	20	31	34	41	2	104	0.00%	94.6/sec	38.90	22.55
HTTP Requ...	339669	20	20	31	34	41	2	104	0.00%	94.6/sec	41.03	13.86
TOTAL	1700353	20	20	31	34	41	2	105	0.00%	472.6/sec	211.10	95.17

Aggregate Report

The data represents the total system load processed during the 1-hour test duration. A total of **1,700,353 samples** were executed with a **0.00% error rate** across all endpoints.

Key Observation: The system maintained an average response time of **20ms** for all critical paths, including authentication, reservation creation, and table availability searches. The throughput peaked at **472.6 requests per second**, demonstrating high concurrent processing capability.

Behavioral Trends



Response Time Graph

The graph illustrates the relationship between the scaling load and system responsiveness over the testing timeline.

Graph Analysis:

- **Initialization Phase:** High volatility is visible in the first few minutes as threads begin their ramp-up.
- **Stability Phase:** Between the 15:24 and 15:52 timestamps, the system reached a "Steady State," maintaining response times between **18ms and 20ms**.
- **Peak Load Performance:** Even as the test reached its final phase (after 15:54), the system settled into a consistent performance band of **22ms**, proving it can handle sustained 1,000-user pressure without spiraling latencies.

Conclusion & Recommendations

The performance test for the 1,000-user load scenario is classified as **PASS**.

The system successfully maintained 100% availability with zero errors throughout the 60-minute duration. Furthermore, the average response time (20ms) outperformed the SRS requirements significantly. Based on these results, the current API implementation is deemed production-ready for the specified load capacity.

Security Test Specification Report

Security Test Case Matrix

This table summarizes the core security test cases, their objectives, and the technical layer they protect.

Test Case ID	Description	Security Layer	Expected Outcome
TC-SEC-001	Password rejected — 7 characters (Min boundary)	Backend + Frontend	400 Bad Request
TC-SEC-005	Password rejected — Missing uppercase/number	Backend + Frontend	400 Bad Request
TC-SEC-006	Account locked after 5 failed login attempts	Backend (DB + Service)	423 Locked
TC-SEC-007	API rejects expired JWT token	Backend Middleware	401 Unauthorized
TC-SEC-008	SQL Injection prevention on login fields	Backend (Prisma ORM)	Injection strings ignored
TC-SEC-009	XSS injection in "Special Request" sanitized	Backend (Helmet.js)	Script tags encoded/stripped
TC-SEC-010	Rate limiting protection (DDoS prevention)	Backend Middleware	429 Too Many Requests
TC-SEC-014	Role-Based Access Control (RBAC) verification	Backend (Authorize)	403 Forbidden for invalid roles

TC-SEC-017	Cryptographic Hashing (Password Protection)	Backend (bcrypt)	Data stored as hash, not text
TC-SEC-021	Insecure Direct Object Reference (IDOR) check	Backend (Ownership)	404 Not Found for unauthorized IDs

---

Detailed Security Implementations

A. Authentication & Password Policy

Vulnerability Prevention: Ensures all passwords meet complexity requirements before database entry.

- **Validation Logic:** Password length 8–20 characters, containing at least 1 uppercase and at least 1 number.
- **Technical Implementation:** Utilizes a PASSWORD_REGEX on both React frontend hooks and Express.js backend middleware.
- **Hashing:** Passwords are never stored in plain text; they are hashed using bcrypt with 12 salt rounds.

B. Session Management & API Security

Vulnerability Prevention: Protects against unauthorized access and brute-force attacks.

- **Account Lockout:** Accounts are suspended for 15 minutes after 5 consecutive failed login attempts.
- **JWT Security:** Tokens are strictly configured with a 1-week expiration (exp) and verified by middleware on every request.
- **Rate Limiting:** Restricted to a maximum of 100 requests per IP per minute using express-rate-limit.

C. Data Integrity & Ownership (IDOR/XSS/SQLi)

Vulnerability Prevention: Ensures users can only access their own data and prevents malicious script execution.

- **SQLi Protection:** Uses Parameterized Queries through Prisma ORM to prevent raw SQL concatenation.
- **XSS Protection:** Implements Helmet.js to set secure HTTP headers and a Content Security Policy (CSP) that denies inline scripts.
- **IDOR/Ownership Validation:** The backend cross-checks the resource owner's ID against the userId extracted from the authenticated JWT token before returning data.

Security Conclusion

The security implementation follows a **"Defense in Depth"** strategy, applying validation at both the Frontend and Backend layers. By combining cryptographic hashing, rigorous RBAC, and middleware-based protections (Rate Limiting, Helmet, JWT), the system is hardened against common OWASP vulnerabilities such as SQL Injection, Cross-Site Scripting, and Broken Access Control.

Team Members & Roles

Student ID	Name	Role
6733226921	Worachart Poungtabtim	QA Lead / BA
6633014021	Kongpop Boonma	Test Case Designer
6733288821	Aphiwich Sangpet	API Tester
6733169821	Patcharapon Srisuwan	Automated Tester
6733065121	Nattarat Samartkit	Performance Tester
6733289421	Apiwit Saengangkhanawin	Security Tester